

PROJECT DOCUMENTATION

Degree Program ECI
ASSIST HEIDI

Bluetooth Gaming Module

By: Kerem Kaan Özmenteşe, Sandeep Sharma, Stiina Mäkelä

Supervisor: Martin Deinhofer, Alija Sabic, Sarah Langer

26.06.2026, Vienna

- **Project Title:** Project Unwired: An Acceleration-Based, Dual-Node Wireless Assistive Controller for Wheelchair Integration
- **Short Description:** A wireless, modular assistive gaming station utilizing an acceleration-tracking joystick and a Bluetooth-based knee switch to enable independent PC gaming for a user with Cerebral Palsy.
- **Team Name:** Team Probello
- **Flag / Logo:**



- **License:** MIT License
- **Estimated Cost of Solution:** €35.00



Figure 1. Final product mounted on wheelchair

Table of Contents

1	Introduction	4
2	Methods	6
2.1	3D Design	9
2.2	Code Design	9
3	Results – Testing	10
3.1	Hardware and Code Integration Testing	10
3.2	Joystick Design Collaboration	10
3.3	Button Mount and Puck.js Integration	10
3.4	Device Operating Principles	10
4	Discussion	11
4.1	Future Improvements	11
4.2	Dongle Decision	11
	References	12

1 Introduction

The objective of this project was to refine and optimize an existing assistive gaming station prototype developed for our co-designer, Natascha. The target games for this system are titles such as Blue Prince, an adventure, strategy, and puzzle-roguelike game played on PC, which is representative of a broader category of games requiring two simultaneous forms of input: camera perspective control, achieved through continuous mouse movement, and character navigation, achieved through discrete directional input via the WASD keys. Enabling fluid, responsive control over both input modalities was therefore a central requirement for the system to be genuinely usable for recreational gaming.

A previous iteration of the assistive gaming station successfully demonstrated that these keybinds could be mapped onto an accessible hardware layout suited to Natascha's needs. However, while functionally adequate, the final design of this earlier prototype introduced several physical and practical challenges that limited its everyday usability. The system was bulky and cumbersome, making it difficult to integrate cleanly into Natascha's existing wheelchair setup. Its operation relied on wiring spanning across the user's wheelchair setup, creating both a visual and physical obstruction and introducing a risk of accidental disconnection or interference with the chair's primary mobility functions. In addition, the prototype depended on external AAA battery compartments and loose elastic bands to secure its components in place, making the system cumbersome to mount, position, and transport independently, and undermining the goal of providing Natascha with a self-sufficient, easy-to-use setup. Considering these limitations, Natascha requested a streamlined, portable, and clean alternative that eliminated the mechanical clutter and physical bulk associated with the previous design, while preserving its core functionality.

To design an effective solution, the hardware structure had to align closely with the user's physiological realities, rather than being adapted from a generic input device after the fact. Natascha lives with Cerebral Palsy. This condition results in spasticity, involuntary tremors, and severe muscle stiffness. Consequently, conventional PC gaming mice, keyboards, and standard gamepads are inaccessible to her, as they demand a level of fine motor precision and simultaneous multi-button coordination that is incompatible with these physical constraints.

For the system to be both usable and sustainable over extended play sessions, it had to leverage Natascha's existing physical strengths without causing rapid fatigue. Her core retained abilities include functional left-hand dexterity and controlled side-leg and knee movements, both of which informed the choice of input mechanism and its physical placement within the final design.

Together, these constraints framed the engineering problem of designing a physically minimal, fatigue-conscious system that channels Natascha's specific retained motor functions into reliable, low-latency PC input. “**Figure 2.**” shows the logo of Team Probello, the team responsible for this project designed by it’s own members.



Figure 2. Flag for the Team Probello

“**Figure 3.**” presents the block diagram of the overall system, illustrating how the wheelchair-mounted input hardware, the wireless mode-switching module, and the host PC interact to deliver both mouse and keyboard-style control from a single, compact, and self-contained setup.

For wiring diagram, please also check GitHub repository. [1]

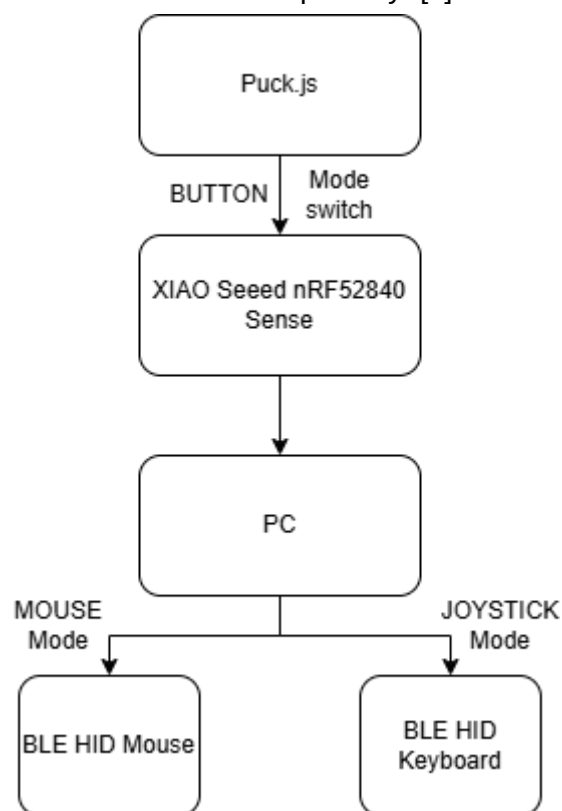


Figure 3. Block diagram for the whole system

2 Methods

To address these constraints, Team Probello, working in close collaboration with the co-designer, brainstormed potential approaches and ultimately engineered a wireless, dual-node assistive setup that eliminates all physical cable connections between the user's hand inputs, leg inputs, and the gaming PC. By treating the hand input and the leg input as two isolated, self-powered wireless units rather than a single tethered assembly, the resulting controller drastically reduces setup time and removes the physical bulk that had previously accumulated on Natascha's wheelchair.

The architecture consists of two main components. The first is the Acceleration-Detecting Joystick, referred to as the Hand Node, which is housed in a custom shell tailored specifically to Natascha's left-hand dexterity. At the core of this node is a Seeed Studio XIAO nRF52840 Sense microcontroller, shown in **Figure 4.**, which integrates a Bluetooth Low Energy radio with an onboard accelerometer and Inertial Measurement Unit (IMU) sensor on a single, compact board.



Figure 4. Photo of the used microcontroller XIAO Seeed Studio nRF52840 Sense

This MCU continuously detects the physical acceleration of the joystick as it is moved and emulates these movements directly as PC Human Interface Device (HID) outputs, namely WASD key presses for character navigation and mouse movement for camera control.

The second component is the Puck.js Switch, referred to as the Knee Node.

Figure 5. shows the Puck.js module used for this purpose, coin-sized Bluetooth Low Energy development board fitted into a custom mechanical fixture originally developed by the previous HEIDI group working with Natascha, the Trentino Team.



Figure 5. Photo of the used button, as a mode switch between MOUSE and KEYBOARD

Positioned directly next to Natascha's wheelchair frame, this fixture is built to respond to her left knee and leg side movements, allowing her to trigger a mode switch using a part of her body distinct from the hand controlling the joystick itself.

“**TABLE 1.**” demonstrates the desired inputs and expected outputs, as associated action for the project.

TABLE 1. Table of controls and associated actions

Physical Input Device	User Action	System Output	Associated Action
Puck.js Knee Switch	Single Press (Left Knee)	Mode Swap Toggle	Cycles between Keyboard Mode and Mouse Mode
Smart Joystick (Keyboard Mode)	Move Left / Right / Forward / Backward	Arrow Keys (W-A-S-D)	Controls character navigation
Smart Joystick (Mouse Mode)	Move Left / Right / Forward / Backward	Mouse Cursor Movement	Controls camera perspective and aiming

The following table provides a comprehensive overview of the Bill of Materials (BOM) and sourcing details required for the assembly of the system. “**TABLE 2.**” illustrates the complete breakdown of the technical components, categorized into primary microcontrollers, power management, wiring interfaces, and essential production tools. Notably, the hardware architecture relies on the Seeed Studio XIAO nRF52840 and a Puck.js Bluetooth smart beacon to handle the core processing and connectivity.

TABLE 2. BOM list for the components on the project

Item	Part Description / Notes	Qty	Manufacturer / Model	Remarks
1	Microcontroller (MCU) : Features integrated Bluetooth LE, internal LiPo charging circuit, and Type-C port.	1	Seeed Studio XIAO nRF52840	
2	Knee Node Processor : Bluetooth Smart Beacon.	1	Puck.js	
3	Power Source : 3.7V / 170mAh rechargeable LiPo battery.	1	HWE601525	
4	Charging Extension : USB-C Male-to-Female Extension Cable (creates a rugged exterior charging socket).	1	-	
5	Toggle Interface : SPST Sub-miniature slide switch.	1	-	
6	Interconnects : Thin-gauge internal hookup wire (fitted with a quick-disconnect plug for clean teardowns).	1	-	
7	Tools Required : Soldering iron, wire strippers, heat-shrink tubing, glue.	-	-	Production Tool Only
8	Joystick Cap : Inherited for core mechanical alignment.	1	Reused Asset	Group Trentino
9	Silicone Rubber Cover : Provides waterproofing and sweat resistance for the joystick interface.	1	Reused Asset	Group Trentino
10	Wheelchair Frame Mount : Side holder retained to clamp the knee node securely to the frame.	1	Reused Asset	Group Trentino

“Figure 5.” Illustrates the housing for the knee button. Note that Puck.js button will be placed on this, in order to keep it stable and let Natascha to switch between MOUSE and KEYBOARD modes instantly.

“Figure 6.” and **“Figure 7.”** shows the whole materials, including MCU XIAO Seeed Studio nRF52840 Sense, Joystick cap, Joystick body, USB-C extension cable, Battery and Switch.



Figure 6. Housing for the knee button.
components (Puck.js)

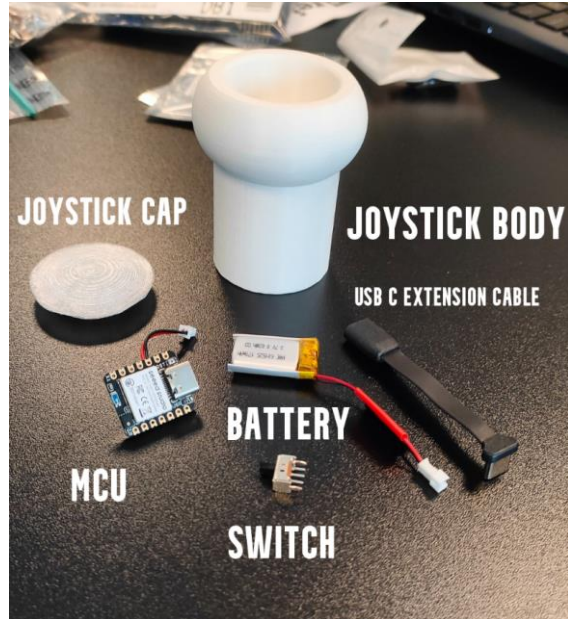


Figure 7. Joystick body along with its
(without the silicone cap)

2.1 3D Design

For the 3D Design, we used Autodesk Fusion 360 2026 along with PrusaSlicer 2.9.5. For more details please see : the GitHub repository path CAD Files/Readme. [1]

2.2 Code Design

The firmware for the two wireless nodes in this system was developed using two distinct toolchains, reflecting the different hardware platforms used.

The code running on the Puck.js module (Knee Node) was developed using the Espruino Web IDE, a browser-based development environment specifically designed for Espruino-powered boards such as the Puck.js [2]. Since this IDE communicates with the Puck.js directly over Bluetooth Low Energy rather than a wired serial connection, it requires the use of a Bluetooth-enabled web browser, such as Google Chrome or Microsoft Edge, to access the development environment and upload code to the device.

The code running on the Seeed Studio XIAO nRF52840 Sense (Hand Node) was developed using the Arduino IDE, leveraging the Seeed XIAO board support package along with the

Adafruit Bluefruit BLE library to implement the dual-role Bluetooth architecture and HID report generation.

All source code for both nodes has been uploaded to a public GitHub repository [1], where it can be freely accessed, reviewed, and reproduced by others wishing to replicate or build upon this project.

3 Results – Testing

3.1 Hardware and Code Integration Testing

The hardware — and especially the interoperability of the code — was tested multiple times as the project progressed. At first, we faced challenges establishing the Bluetooth connection, but as we researched the issue and studied various sources, we eventually got the Bluetooth link working reliably.

3.2 Joystick Design Collaboration

We collaborated with Natacha on the joystick design, and the final result turned out pleasing. Later, we discovered a measurement error in the 3D model that now prevents the use of a silicone tube between the wheelchair's joystick controller and our custom joystick. The silicone tube would have provided a better fit and prevented the joystick from rotating or detaching. Instead, we decided to use a wide elastic band that just fits between the wheelchair's control pin and our joystick.

3.3 Button Mount and Puck.js Integration

The previous team had designed an excellent holder for the knee-operated button, and we found that the original silicone case of the Puck.js fits perfectly into that holder. Therefore, we only needed to update the Puck.js code to make it function as intended.

3.4 Device Operating Principles

The joystick itself acts as a keyboard:

- Moving it **up** presses **W**
- Moving it **down** presses **S**
- Moving it **right** presses **D**
- Moving it **left** presses **A**

When the Puck.js button is pressed with the knee, the joystick switches to **mouse functionality**. We noticed that Natacha's wheelchair control unit is slightly tilted downward, so we had to take that into account when adjusting the sensitivity for both keyboard and mouse modes.

4 Discussion

4.1 Future Improvements

In the future, it would be beneficial to redesign the joystick's housing so that the hole placed over the wheelchair's control pin could be **triangular or square** instead of round. Natacha has already confirmed with the wheelchair manufacturer that the top of the control pin can be slightly sanded to help the joystick stay firmly in place.

4.2 Dongle Decision

Originally, we also intended to use a dongle in our system, but we encountered difficulties programming it and integrating it with the other devices we were using. We therefore decided to leave it out of the project entirely. The dongle's original purpose was to reduce Bluetooth latency, but we found that the system performs fast enough even without it.

References

- [1] C.-G.-S. /. T. Probello, „GitHub,“ 27 06 2026. [Online]. Available: <https://github.com/inclusion-international/Custom-Game-Station>.
- [2] Espruino, „Espruino Web IDE,“ [Online]. Available: <https://www.espruino.com/ide/>. [Zugriff am 27 06 2026].
- [3] Espruino, „Puck.js Software Reference,“ Espruino, 27 06 2026. [Online]. Available: <https://www.espruino.com/ReferencePUCKJS>.
- [4] S. Studio, „https://wiki.seeedstudio.com/XIAO_BLE/,“ Seeed Studio, 27 06 2026. [Online]. Available: https://wiki.seeedstudio.com/XIAO_BLE/.

https://wiki.seeedstudio.com/XIAO_BLE/

<https://www.puck-js.com/>